

VÉLØ ORACLE PRIME — TECHNICAL DEEP DIVE

How VÉLØ Actually Works: From Raw Data to Strike Decision

Complete System Architecture, Core Predictor Mechanics, and Decision Framework

TABLE OF CONTENTS

1. [Architecture Overview](#)
 2. [Layer 1: The 61+ Feature Engines](#)
 3. [Layer 2: The Core Predictor \(LightGBM/XGBoost\)](#)
 4. [Layer 3: RPD-C v2 Tag System](#)
 5. [Layer 4: Market Constraint Engine](#)
 6. [Layer 5: Intent Detection Engine](#)
 7. [Layer 6: Scenario Evidence Gate](#)
 8. [Layer 7: Quarantine Gates](#)
 9. [Layer 8: Final Decision Policy](#)
 10. [The Doctrine: Why VÉLØ Works](#)
 11. [Complete Example: Race Walkthrough](#)
-

ARCHITECTURE OVERVIEW

VÉLØ is **not a single prediction model**. It's a **multi-layered intelligence system** that combines 61+ feature engines, behavioral analysis, market psychology, and strategic doctrine into a unified decision framework.

The Flow

```
RAW INPUTS (race card, market data, historical data)
  ↓
61+ FEATURE ENGINES (parallel micro-analyzers)
  ↓
FEATURE VECTOR [ $f_1$ ,  $f_2$ ,  $f_3$ , ...,  $f_{61}$ ]
  ↓
CORE PREDICTOR (Gradient Boosting: LightGBM/XGBoost)
  ↓
PROBABILITIES: p(win), p(top4), confidence
  ↓
STRATEGIC INTELLIGENCE PACK V2
├─ RPD-C v2 Tag System (T/H/P/S/E classification)
├─ Market Constraint Engine (BSP drift analysis)
├─ Intent Detection Engine (trainer "go day" signals)
├─ Scenario Evidence Gate (pattern validation)
└─ Quarantine Gates (uncertainty filters)
  ↓
DECISION POLICY (chassis selection)
  ↓
FINAL VERDICT: Top Strike / Value / Danger / Suppress / Quarantine
```



LAYER 1: THE 61+ FEATURE ENGINES

What They Are

Feature engines are **pure functions** that transform raw racing data into **signals**. Each engine is a micro-analyzer that looks at one specific aspect of a race.

Key Properties:

- **Deterministic:** Same inputs → same output
- **Independent:** Each engine runs in isolation
- **Ablation-testable:** Can be removed to test robustness
- **Parallel-executable:** All 61 can run simultaneously

The 61 Features (Organized by Domain)

1. Form Domain (10 features)

Feature	Description	Example Value
rpr_last_3_avg	Average Racing Post Rating over last 3 runs	95.3
ts_last_3_avg	Average Topspeed over last 3 runs	88.7
or_current	Current Official Rating	82
form_consistency_score	Standard deviation of last 5 RPRs (lower = more consistent)	4.2
peak_form_recency	Days since best RPR in last 12 months	28
form_decline_flag	Binary: Is form declining over last 3 runs?	0 (no)
class_drop_indicator	Numerical: Class drop magnitude (positive = dropping)	+2
career_win_rate	Win percentage across career	0.18
c_d_win_rate	Win rate at this course/distance	0.33
recent_placings	Top-3 finishes in last 5 runs	3

2. Pace Domain (8 features)

Feature	Description	Example Value
early_pace_score	Early speed capability (0-100)	72
late_pace_score	Finishing speed capability (0-100)	85
pace_geometry	How pace profile fits race shape (-1 to +1)	0.65
sectional_speed_variance	Consistency of sectional times	2.1
pace_collapse_probability	Likelihood of front-runners tiring (0-1)	0.42
closer_advantage_score	Does pace setup favor closers? (0-100)	68
front_runner_risk	Likelihood of burning out (0-1)	0.28
stamina_stretch_threshold	Can horse handle distance? (0-1)	0.91

3. Draw Domain (5 features)

Feature	Description	Example Value
draw_bias_score	Historical advantage/disadvantage of this draw (-100 to +100)	+12
draw_advantage_index	Normalized draw position (0-1, 0=worst, 1=best)	0.78
rail_position_flag	Binary: Is horse on the rail?	1 (yes)
wide_draw_penalty	Disadvantage from wide draw (0-50)	5
draw_going_interaction	How draw advantage changes with going (-1 to +1)	0.35

4. Trainer/Jockey Domain (12 features)

Feature	Description	Example Value
trainer_strike_rate_14d	Trainer win rate last 14 days	0.22
trainer_strike_rate_90d	Trainer win rate last 90 days	0.18
jockey_strike_rate_14d	Jockey win rate last 14 days	0.25
jockey_strike_rate_90d	Jockey win rate last 90 days	0.21
trainer_jockey_combo_win_rate	Win rate when this pair team up	0.28
trainer_course_win_rate	Trainer's record at this track	0.19
jockey_course_win_rate	Jockey's record at this track	0.23
first_choice_jockey_flag	Binary: Is this the trainer's #1 jockey?	1 (yes)
jockey_switch_signal	Binary: Did jockey change from last run?	1 (yes)
claimer_flag	Binary: Is a claiming jockey used?	0 (no)
trainer_intent_score	Intent Engine output (0-100)	72
stable_form_cycle	Is stable in form? (0-100)	65

5. Course/Going/Distance Domain (10 features)

Feature	Description	Example Value
course_form_score	Horse's record at this track (0-100)	78
going_suitability_score	How well horse handles this going (0-100)	82
distance_win_rate	Win rate at this distance	0.24
distance_optimal_flag	Binary: Is this horse's best distance?	1 (yes)
going_change_impact	How going change affects horse (-1 to +1)	0.15
track_bias_alignment	Does running style suit track bias? (0-100)	68
surface_suitability	AW vs turf preference (0-100)	85
course_distance_combo_score	C&D record (0-100)	72
trip_suitability_index	Overall trip match (0-100)	79
going_extreme_flag	Binary: Is going heavy/firm?	0 (no)

6. Class Domain (6 features)

Feature	Description	Example Value
class_rating	Numerical class level (1-7)	4
class_movement	Moving up/down in class? (-3 to +3)	-1 (dropping)
class_drop_flag	Binary: Dropping in class?	1 (yes)
class_rise_flag	Binary: Rising in class?	0 (no)
class_competitive_index	Can horse compete at this level? (0-100)	75
or_vs_class_gap	Official Rating vs class requirement	+8

7. Recency Domain (4 features)

Feature	Description	Example Value
days_since_last_run	Layoff duration	21
runs_this_season	Number of runs this season	5
layoff_flag	Binary: Is layoff >90 days?	0 (no)
peak_fitness_window	Binary: In optimal fitness window (14-35 days)?	1 (yes)

8. Weight/Age Domain (3 features)

Feature	Description	Example Value
weight_carried	Actual weight in kg	59.5
age	Horse age in years	5
weight_for_age_adjustment	Age-based weight adjustment	-2.5

9. Market Domain (3 features)

Feature	Description	Example Value
odds_decimal	Current odds (decimal format)	4.5
odds_drift_30m	Odds movement in last 30 minutes (%)	-8.2
bsp_advantage	BSP vs SP gap (%)	+12.5



LAYER 2: THE CORE PREDICTOR (LightGBM/XGBoost)

What Is Gradient Boosting?

Gradient Boosting is a machine learning technique that builds an ensemble of **weak learners** (decision trees) that work together to make predictions.

How It Works: The Intuition

Imagine you're trying to predict a horse's win probability. You ask 100 experts, but each expert is only slightly better than random guessing (hence "weak learners").

The trick: Each new expert focuses on **correcting the mistakes** of the previous experts.

Process:

1. **Expert 1** makes predictions → gets 60% accuracy
2. **Expert 2** looks at what Expert 1 got wrong → corrects those mistakes
3. **Expert 3** looks at what Experts 1+2 still get wrong → corrects those
4. Repeat 100 times
5. **Final prediction** = weighted average of all 100 experts

The Mathematics (Simplified)

Step 1: Initialize with a baseline prediction

$$F_0(x) = \text{mean}(y)$$

For horse racing, this might be $F_0 = 0.10$ (10% win probability, assuming 10-horse field).

Step 2: For each iteration $m = 1$ to M (e.g., $M = 100$ trees):

A. Calculate residuals (errors from previous prediction):

$$r_{im} = y_i - F_{m-1}(x_i)$$

Where:

- y_i = actual outcome (1 if won, 0 if lost)
- $F_{m-1}(x_i)$ = current prediction

- r_{im} = error (residual)

B. Fit a decision tree $h_m(x)$ to predict the residuals:

The tree learns patterns in the 61 features that explain why the previous prediction was wrong.

C. Update the model:

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$$

Where:

- η = learning rate (e.g., 0.1) — controls how much each tree contributes
- $h_m(x)$ = new tree's prediction

Step 3: Final prediction after M iterations:

$$F(x) = F_0 + \eta \cdot \sum h_m(x)$$

This is the sum of all trees, each correcting the previous ones.

Concrete Example: Predicting Win Probability

Race: 10 horses, we're predicting Horse A

Iteration 0 (Baseline):

$$F_0 = 0.10 \text{ (10\% baseline for 10-horse field)}$$

Iteration 1:

- **Tree 1** looks at features: $rpr_last_3_avg=95$, $first_choice_jockey=1$, $c_d_winner=1$
- **Tree 1 says:** "High RPR + first-choice jockey + C&D winner → add +0.15"
- **Updated prediction:** $F_1 = 0.10 + 0.1 \cdot 0.15 = 0.115$ (11.5%)

Iteration 2:

- **Actual outcome:** Horse A won ($y = 1$)
- **Residual:** $r = 1 - 0.115 = 0.885$ (we're still way off)
- **Tree 2** looks at features: `trainer_strike_rate_14d=0.25, stable_form_cycle=75`
- **Tree 2 says:** "Trainer in hot form + stable firing → add +0.20"
- **Updated prediction:** $F_2 = 0.115 + 0.1 * 0.20 = 0.135$ (13.5%)

Iteration 3:

- **Residual:** $r = 1 - 0.135 = 0.865$
- **Tree 3** looks at features: `draw_bias_score=+12, pace_geometry=0.65`
- **Tree 3 says:** "Good draw + pace setup suits → add +0.18"
- **Updated prediction:** $F_3 = 0.135 + 0.1 * 0.18 = 0.153$ (15.3%)

... continue for 100 iterations ...

Final prediction after 100 trees:

$$F_{100}(x) = 0.35 \text{ (35\% win probability)}$$

Why LightGBM/XGBoost?

Both are **optimized implementations** of gradient boosting with key advantages:

LightGBM Advantages:

1. **Leaf-wise tree growth** — grows trees by splitting the leaf with maximum gain (faster, more accurate)
2. **Histogram-based splitting** — bins continuous features into discrete bins (faster training)
3. **Categorical feature support** — handles categorical features (trainer, jockey, course) natively
4. **GPU acceleration** — can train on GPU for massive speed-ups

5. **Low memory usage** — efficient for large datasets

XGBoost Advantages:

1. **Regularization** — L1/L2 penalties prevent overfitting
2. **Weighted quantile sketch** — handles sparse data well
3. **Parallel processing** — builds trees in parallel
4. **Cross-validation built-in** — easy hyperparameter tuning
5. **Feature importance** — shows which features matter most

VÉLØ's Implementation Choice

VÉLØ uses LightGBM for:

- **Speed:** Trains 10x faster than XGBoost on 61 features
- **Categorical handling:** Trainer/jockey/course features handled natively
- **Leaf-wise growth:** Better accuracy on complex racing patterns

Training the Core Predictor

Training Data Structure:

race_id	horse	f ₁	f ₂	...	f ₆₁	won	top4
R001	A	95	88	...	4.5	1	1
R001	B	82	75	...	6.2	0	1
R001	C	78	71	...	8.5	0	1
...	...	...	...	...	...	...	...

Training targets:

- `won` — Binary (1 if won, 0 if lost)
- `top4` — Binary (1 if finished top-4, 0 otherwise)

Hyperparameters (Example):

```
params = {  
    'objective': 'binary', # Binary classification  
    'metric': 'auc', # Area Under Curve (model quality)  
    'boosting_type': 'gbdt', # Gradient Boosting Decision Tree  
    'num_leaves': 31, # Max leaves per tree  
    'learning_rate': 0.05, # How much each tree contributes  
    'feature_fraction': 0.8, # Use 80% of features per tree  
    'bagging_fraction': 0.8, # Use 80% of data per tree  
    'bagging_freq': 5, # Bagging every 5 iterations  
    'max_depth': 6, # Max tree depth  
    'min_data_in_leaf': 20, # Min samples per leaf  
    'lambda_l1': 0.1, # L1 regularization  
    'lambda_l2': 0.1, # L2 regularization  
}
```

Training Process:

```
import lightgbm as lgb  
  
# Load training data  
train_data = lgb.Dataset(X_train, label=y_train)  
val_data = lgb.Dataset(X_val, label=y_val, reference=train_data)  
  
# Train model  
model = lgb.train(  
    params,  
    train_data,  
    num_boost_round=1000, # 1000 trees  
    valid_sets=[train_data, val_data],  
    early_stopping_rounds=50, # Stop if no improvement for 50 rounds  
    verbose_eval=100  
)  
  
# Make predictions  
predictions = model.predict(X_test)
```

Model Output

For each horse, the model outputs:

```
{
  'p_win': 0.35, # 35% probability of winning
  'p_top4': 0.72, # 72% probability of finishing top-4
  'confidence': 85 # Model certainty (0-100)
}
```

Confidence calculation:

```
def calculate_confidence(p_win, feature_vector):
    # Factors that increase confidence:
    # 1. High probability (p_win > 0.3)
    # 2. Strong features (high RPR, first-choice jockey, etc.)
    # 3. Low feature variance (consistent signals)

    confidence = 50 # Baseline

    if p_win > 0.3:
        confidence += 20
    if feature_vector['first_choice_jockey'] == 1:
        confidence += 10
    if feature_vector['rpr_last_3_avg'] > 90:
        confidence += 10
    if feature_vector['form_consistency_score'] < 5:
        confidence += 10

    return min(confidence, 100)
```

Feature Importance

After training, VÉLØ can inspect which features matter most:

```
importance = model.feature_importance(importance_type='gain')
```

```
# Top 10 features by importance:  
# 1. rpr_last_3_avg (18.2%)  
# 2. trainer_intent_score (12.5%)  
# 3. first_choice_jockey_flag (9.8%)  
# 4. going_suitability_score (8.3%)  
# 5. draw_bias_score (7.1%)  
# 6. pace_geometry (6.4%)  
# 7. class_drop_indicator (5.9%)  
# 8. c_d_win_rate (5.2%)  
# 9. bsp_advantage (4.8%)  
# 10. stable_form_cycle (4.3%)
```

Key insight: Form (RPR) is important, but **intent** and **jockey choice** are nearly as important.



LAYER 3: RPD-C v2 TAG SYSTEM

What RPD-C Does

RPD-C (Runner Profile & Disposition - Calibrated v2) is a **rule-based tagging system** that classifies each runner based on **evidence patterns**.

Purpose: Filter out noise and focus on horses with genuine winning chances.

The Tags

Tag	Meaning	Interpretation	Action
T	Target	Strong winning chance	Consider for strike
H	Hold	Solid contender, not standout	Monitor, backup pick
P	Prep	Not ready to win today	Exclude from consideration
S	Swerve	Avoid, poor match	Eliminate
E	Eliminate	No chance	Hard eliminate

Evidence Patterns

T (Target) — 4+ Positive Signals Required

Positive signals:

1. **Peak form** — RPR within 5 of career best
2. **First-choice jockey** — Trainer's #1 rider booked
3. **Course winner** — Won at this track before
4. **Distance winner** — Won at this distance before
5. **Class match** — OR matches class requirement
6. **Optimal trip** — Distance/going suit horse
7. **Good draw** — Draw bias score >0
8. **Stable in form** — Trainer has recent winners
9. **Progressive** — Improving form trend
10. **Gear addition** — First-time blinkers/visor

Example:

```
runner = {  
  'rpr_last_3_avg': 95, # ✓ Peak form  
  'first_choice_jockey': True, # ✓ First-choice jockey  
  'c_d_winner': True, # ✓ Course & distance winner  
  'draw_bias_score': 12, # ✓ Good draw  
  'stable_form_cycle': 75, # ✓ Stable in form  
}  
# 5 positive signals → Tag = T (Target)
```

H (Hold) — 3 Positive Signals

Example:

```
runner = {
    'rpr_last_3_avg': 88, # ✓ Solid form
    'course_winner': True, # ✓ Course winner
    'optimal_trip': True, # ✓ Optimal trip
    'first_choice_jockey': False, # x Not first-choice
}
# 3 positive signals → Tag = H (Hold)
```

P (Prep) — 2+ Prep Indicators

Prep indicators:

1. **Long layoff** — >90 days since last run
2. **New yard** — First run for new trainer
3. **Wind surgery** — Recent breathing operation
4. **Wrong trip** — Distance/going don't suit
5. **Jockey downgrade** — Worse jockey than usual
6. **Class rise** — Stepping up in class
7. **First-time headgear** — Testing new equipment

Example:

```
runner = {
    'days_since_last': 120, # ✓ Long layoff
    'new_yard': True, # ✓ New trainer
    'wind_surgery': True, # ✓ Recent surgery
}
# 3 prep indicators → Tag = P (Prep)
```

CRITICAL LEARNING (from American State):

P tag should be split into:

- **P1 (Prep)** — Not ready, avoid
- **P2 (Reactivation)** — Ready to fire after interventions

Reactivation signals:

- Wind surgery (problem **fixed**, not created)
- New yard (fresh start, not decline)
- Break <120 days (sufficient recovery)
- BSP advantage >15% (informed money recognizes value)

S (Swerve) — 2+ Negative Signals

Negative signals:

1. **Poor form** — RPR declining over last 3 runs
2. **Wrong going** — Going suitability score <40
3. **Unsuitable course** — Course form score <30
4. **Wide draw penalty** — Draw bias score <-10
5. **Jockey downgrade** — Worse jockey than usual
6. **Class rise** — Stepping up in class
7. **Long layoff** — >180 days since last run

Example:

```
runner = {
  'form_decline_flag': True,  # ✓ Poor form
  'going_suitability_score': 35,  # ✓ Wrong going
  'class_rise_flag': True,  # ✓ Class rise
}
# 3 negative signals → Tag = S (Swerve)
```

E (Eliminate) — 2+ Hard Elimination Signals

Hard elimination signals:

1. **Outclassed** — OR 20+ below class requirement
2. **Wrong distance** — Distance win rate = 0
3. **Never won** — Career win rate = 0 (in non-maidens)
4. **Extreme going mismatch** — Going suitability <20
5. **Trainer/jockey combo fail** — 0% win rate together

6. Extreme draw disadvantage — Draw bias <-20

Example:

```
runner = {  
  'or_vs_class_gap': -25, # ✓ Outclassed  
  'career_win_rate': 0.0, # ✓ Never won  
}  
# 2 hard elimination signals → Tag = E (Eliminate)
```

Tag Distribution (Typical Race)

10-horse handicap:

- **T (Target):** 2-3 horses
- **H (Hold):** 2-3 horses
- **P (Prep):** 1-2 horses
- **S (Swerve):** 2-3 horses
- **E (Eliminate):** 1-2 horses

VÉLØ only considers T and H tags for strike recommendations.

LAYER 4: MARKET CONSTRAINT ENGINE

The Problem: Fake Favourites

Fake favourites are horses that the public backs heavily but have **no genuine chance**. They're created by:

1. **Narrative hype** — Media stories, big names
2. **Recency bias** — Won last time, so must win again
3. **Trainer reputation** — Big-name trainer = automatic favourite
4. **Market manipulation** — Syndicates artificially shortening odds
5. **Public ignorance** — Casual bettors follow the crowd

Result: Overbet favourites with poor value.

How VÉLØ Detects Fake Favourites

The **Market Constraint Engine** applies a **hard gate** to favourites using **BSP drift analysis**.

BSP Drift Logic

BSP (Betfair Starting Price) = price at which the race starts on Betfair (informed money)

SP (Starting Price) = official starting price (public money)

Key Insight: If a horse **drifts** from BSP to SP, it means:

- **Informed money** (professionals) bet early at higher odds
- **Public money** (casuals) bet late at lower odds
- The horse is **overbet** (fake favourite)

BSP Drift Calculation:

```
bsp_drift_pct = ((sp - bsp) / bsp) * 100

# Example 1: Fake favourite
# BSP = 3.0, SP = 2.5
# Drift = ((2.5 - 3.0) / 3.0) * 100 = -16.7%
# Interpretation: Drifted 16.7% (shortened in public betting)

# Example 2: Genuine favourite
# BSP = 2.8, SP = 2.6
# Drift = ((2.6 - 2.8) / 2.8) * 100 = -7.1%
# Interpretation: Slight shortening, within normal range
```

The Hard Gate

Trigger conditions:

1. Horse is the **favourite** (lowest odds)
2. BSP drift **< -15%** (shortened 15%+ in public betting)

3. **3+ counter-signals** present (evidence against the favourite)

Counter-signals:

1. **Long layoff** — >90 days since last run
2. **Class rise** — Stepping up in class
3. **Wrong going** — Going suitability score <50
4. **Poor course record** — Course form score <40
5. **Jockey downgrade** — Not first-choice jockey
6. **Heavy market drift** — BSP drift < -20%
7. **No recent form** — No run in last 60 days
8. **Prep indicators** — Tagged as P (Prep)

Decision logic:

```

def check_market_constraint(runner):
    if not runner['is_favourite']:
        return "PASS" # Not a favourite, no constraint

    bsp_drift = ((runner['sp'] - runner['bsp']) / runner['bsp']) * 100

    if bsp_drift >= -15:
        return "PASS" # Drift within acceptable range

    # Count counter-signals
    counter_signals = 0
    if runner['days_since_last'] > 90:
        counter_signals += 1
    if runner['class_rise_flag']:
        counter_signals += 1
    if runner['going_suitability_score'] < 50:
        counter_signals += 1
    if runner['course_form_score'] < 40:
        counter_signals += 1
    if not runner['first_choice_jockey']:
        counter_signals += 1
    if bsp_drift < -20:
        counter_signals += 1
    if runner['days_since_last'] > 60:
        counter_signals += 1
    if runner['rpd_tag'] == 'P':
        counter_signals += 1

    if counter_signals >= 3:
        return "DISMISS_FAVOURITE" # Hard gate triggered

    return "PASS"

```

Example: Detecting a Fake Favourite

Race: Wolverhampton 19:30

Favourite: Mr Nugget ($\frac{5}{2}$)

- BSP: 2.6
- SP: 2.5
- Drift: -3.8% (slight shortening, acceptable)

- Counter-signals: 1 (hat-trick bid pressure)
- **Verdict:** RESPECT FAVOURITE (drift <15%, counter-signals)

Ignored horse: American State (17.0)

- BSP: 21.0
- SP: 17.0
- Drift: +19% (lengthened, informed money took value)
- **Verdict:** BSP advantage indicates informed money (should have been flagged)

Post-race learning: American State won @ 17.0. VÉLØ missed the **BSP advantage signal** because it was tagged P (Prep) instead of R (Reactivation).

BSP Advantage (Reverse Logic)

BSP advantage = when a horse **lengthens** from BSP to SP (positive drift).

Interpretation:

- Informed money bet early at **shorter odds**
- Public ignored the horse
- **Value opportunity**

Example:

```
# Horse: American State
# BSP: 21.0, SP: 17.0
# BSP advantage = ((17.0 - 21.0) / 21.0) * 100 = +19%
# Interpretation: Informed money recognized value, public didn't
```

New rule (post-SIGMA):

```
if runner['rpd_tag'] == 'P' and bsp_advantage > 15:
    # Reclassify as P2 (Reactivation), not P1 (Prep)
    runner['rpd_tag'] = 'P2'
    runner['consider_for_strike'] = True
```

LAYER 5: INTENT DETECTION ENGINE (T.I.E.)

What Intent Detection Does

T.I.E. (Trainer Intention Engine) detects whether a trainer is **seriously trying to win** or just **giving the horse a run**.

Why this matters:

- Trainers don't always try to win every race
- Some runs are **educational** (learning experiences)
- Some runs are **prep** (fitness building)
- Some runs are **genuine attempts** (go days)

VÉLØ needs to know which is which.

Intent Signals

Positive Intent (Go Day) — 8 Signals

Signal	Weight	Evidence
First-choice jockey booked	+20	Trainer's #1 rider
Jockey switch to upgrade	+15	Better jockey than last time
Optimal trip	+15	Distance/going match horse's best
Class drop	+10	Dropping in class to win
Gear additions	+10	First-time blinkers, visor, tongue-tie
Weight tactics	+10	Claiming jockey for weight advantage
Course specialist	+10	Horse has won here before
Stable in form	+10	Trainer has recent winners

Negative Intent (Prep Day) — 5 Signals

Signal	Weight	Evidence
Jockey downgrade	-20	Worse jockey than usual
Wrong trip	-15	Distance/going don't suit
Class rise	-10	Stepping up in class (educational)
Long layoff	-10	First run back after 6+ months
New yard	-10	First run for new trainer (settling in)

Intent Score Calculation

```
def calculate_intent_score(runner):
    intent_score = 0

    # Positive intent signals
    if runner['first_choice_jockey']:
        intent_score += 20
    if runner['jockey_upgrade']:
        intent_score += 15
    if runner['optimal_trip']:
        intent_score += 15
    if runner['class_drop_flag']:
        intent_score += 10
    if runner['gear_addition']:
        intent_score += 10
    if runner['weight_tactics']:
        intent_score += 10
    if runner['c_d_winner']:
        intent_score += 10
    if runner['stable_form_cycle'] > 60:
        intent_score += 10

    # Negative intent signals
    if runner['jockey_downgrade']:
        intent_score -= 20
    if runner['wrong_trip']:
        intent_score -= 15
    if runner['class_rise_flag']:
        intent_score -= 10
    if runner['days_since_last'] > 180:
        intent_score -= 10
    if runner['new_yard']:
        intent_score -= 10

    return intent_score # Range: -50 to +80
```

Intent Score Interpretation

Score Range	Interpretation	Action
60+	High intent (trainer is going for it)	Strong consideration
30-59	Moderate intent	Normal consideration
0-29	Low intent	Weak consideration
<0	Prep run (not trying to win)	Exclude

Example: High Intent Horse

```
runner = {  
  'first_choice_jockey': True,  # +20  
  'optimal_trip': True,  # +15  
  'class_drop_flag': True,  # +10  
  'c_d_winner': True,  # +10  
  'stable_form_cycle': 75,  # +10  
}  
# Intent score = 65 → HIGH INTENT
```

Example: Prep Run

```
runner = {  
  'jockey_downgrade': True,  # -20  
  'class_rise_flag': True,  # -10  
  'days_since_last': 200,  # -10  
}  
# Intent score = -40 → PREP RUN (exclude)
```

LAYER 6: SCENARIO EVIDENCE GATE

What Scenarios Are

Scenarios are **race-winning patterns** that VÉLØ recognizes. Each scenario has a **code** and **required evidence**.

Purpose: Validate that a horse fits a proven winning pattern.

The 8 Core Scenarios

S1: Hat-trick Bid

Pattern: Horse attempting to win 3 races in a row.

Required evidence:

1. Won last 2 races
2. Same class level
3. Same trip (distance/going)
4. <30 days since last win

Success rate: 35% (historically)

Example:

```
runner = {  
    'last_2_results': ['1', '1'], # ✓ Won last 2  
    'class_movement': 0, # ✓ Same class  
    'trip_match': True, # ✓ Same trip  
    'days_since_last': 21, # ✓ <30 days  
}  
# Scenario S1 VALID
```

S2: Course Specialist

Pattern: Horse with dominant record at this course.

Required evidence:

1. 3+ wins at this course
2. Optimal trip (distance/going match)
3. Recent form solid (RPR within 10 of best)
4. First-choice jockey

Success rate: 42% (historically)

Example:

```
runner = {  
    'course_wins': 4, # ✓ 4 wins at this course  
    'optimal_trip': True, # ✓ Optimal trip  
    'rpr_vs_best': -6, # ✓ Within 10 of best  
    'first_choice_jockey': True, # ✓ First-choice jockey  
}  
# Scenario S2 VALID
```

S3: Class Dropper

Pattern: Horse dropping in class to win.

Required evidence:

1. Dropping 2+ classes
2. Recent form solid (RPR within 10 of best)
3. Optimal trip
4. First-choice jockey

Success rate: 38% (historically)

Example:

```
runner = {
  'class_movement': -2, # ✓ Dropping 2 classes
  'rpr_vs_best': -8, # ✓ Within 10 of best
  'optimal_trip': True, # ✓ Optimal trip
  'first_choice_jockey': True, # ✓ First-choice jockey
}
# Scenario S3 VALID
```

S4: Reactivation

Pattern: Horse returning from break with positive interventions.

Required evidence:

1. Break 60-120 days
2. Positive interventions (wind surgery, new yard, gear)
3. First-choice jockey
4. BSP advantage >15%

Success rate: 28% (historically, but underbet)

Example:

```
runner = {
  'days_since_last': 85, # ✓ 60-120 days
  'wind_surgery': True, # ✓ Positive intervention
  'first_choice_jockey': True, # ✓ First-choice jockey
  'bsp_advantage': 19, # ✓ BSP advantage >15%
}
# Scenario S4 VALID (Reactivation)
```

This is the scenario VÉLØ missed with American State.

S5: Pace Collapse Setup

Pattern: Front-runners will tire, closer has late speed.

Required evidence:

1. Pace collapse probability >60%
2. Horse has late pace score >70
3. Closer advantage score >60
4. Optimal trip

Success rate: 32% (historically)

Example:

```
runner = {  
    'pace_collapse_probability': 0.68, # ✓ >60%  
    'late_pace_score': 82, # ✓ >70  
    'closer_advantage_score': 75, # ✓ >60  
    'optimal_trip': True, # ✓ Optimal trip  
}  
# Scenario S5 VALID
```

S6: Draw Advantage

Pattern: Low draw at track with strong rail bias.

Required evidence:

1. Draw bias score >15
2. Rail position (stall 1-3)
3. Optimal trip
4. Recent form solid

Success rate: 36% (historically)

Example:

```
runner = {  
  'draw_bias_score': 18, # ✓ >15  
  'stall': 2, # ✓ Rail position  
  'optimal_trip': True, # ✓ Optimal trip  
  'rpr_vs_best': -7, # ✓ Within 10 of best  
}  
# Scenario S6 VALID
```

S7: Trainer Intent Spike

Pattern: Trainer pulling out all stops to win.

Required evidence:

1. Intent score >65
2. First-choice jockey
3. Optimal trip
4. Stable in form (>60)

Success rate: 40% (historically)

Example:

```
runner = {  
  'intent_score': 72, # ✓ >65  
  'first_choice_jockey': True, # ✓ First-choice jockey  
  'optimal_trip': True, # ✓ Optimal trip  
  'stable_form_cycle': 68, # ✓ >60  
}  
# Scenario S7 VALID
```

S8: Market Misdirection

Pattern: Overbet favourite, value horse ignored.

Required evidence:

1. Favourite has BSP drift <-15%

2. Favourite has 3+ counter-signals
3. This horse has BSP advantage >10%
4. This horse tagged T (Target)

Success rate: 30% (historically, but high value)

Example:

```
favourite = {  
    'bsp_drift': -18, # ✓ <-15%  
    'counter_signals': 4, # ✓ 3+  
}  
runner = {  
    'bsp_advantage': 15, # ✓ >10%  
    'rpd_tag': 'T', # ✓ Target  
}  
# Scenario S8 VALID (Market Misdirection)
```


Scenario Validation Logic

```
def validate_scenario(runner, proposed_scenario):
    required_evidence = SCENARIO_REQUIREMENTS[proposed_scenario]
    evidence_present = []

    for requirement in required_evidence:
        if check_evidence(runner, requirement):
            evidence_present.append(requirement)

    evidence_ratio = len(evidence_present) / len(required_evidence)

    if evidence_ratio >= 0.75: # 75%+ evidence present
        return {
            'verdict': 'VALID',
            'evidence_ratio': evidence_ratio,
            'missing_evidence': [r for r in required_evidence if r not in
evidence_present]
        }
    else:
        return {
            'verdict': 'INSUFFICIENT_EVIDENCE',
            'evidence_ratio': evidence_ratio,
            'missing_evidence': [r for r in required_evidence if r not in
evidence_present]
        }
```



LAYER 7: QUARANTINE GATES

What Quarantine Does

Quarantine means VÉLØ refuses to issue a **strike recommendation** because **uncertainty is too high**.

Doctrine: Truth before optimization.

VÉLØ would rather **issue no recommendation** than issue a **low-confidence recommendation**.

The 5 Quarantine Gates

Q5: Chaos Mode

Trigger: Heavy/soft going + large field (12+ runners)

Reason:

- Heavy/soft going makes form unreliable
- Large fields increase randomness
- Combination = chaos

Example:

```
race = {  
  'going': 'HEAVY',  
  'runners': 15,  
}  
# Q5 TRIGGERED → QUARANTINE
```

Historical data: Win favourite strike rate drops from 35% (good going) to 22% (heavy going, large field).

Q6: Small Field

Trigger: runners

Reason:

- Form becomes less reliable in small fields
- Pace dynamics unpredictable
- Odds often compressed

Example:

```
race = {  
    'runners': 4,  
}  
# Q6 TRIGGERED → QUARANTINE (or conditional strike with low confidence)
```

Q7: No Form Data

Trigger: Maiden race with no prior form

Reason:

- No historical performance data
- Market often misprices maidens
- High uncertainty

Example:

```
race = {  
    'race_type': 'MAIDEN',  
    'runners_with_form': 0,  
}  
# Q7 TRIGGERED → QUARANTINE
```

Q8: Market Chaos

Trigger: Odds swinging wildly, no clear favourite

Reason:

- Market uncertainty indicates information vacuum
- No consensus = high risk

Example:

```
race = {  
    'favourite_odds': 4.5, # No clear favourite (<3.0)  
    'odds_volatility': 25, # High volatility (>20%)  
}  
# Q8 TRIGGERED → QUARANTINE
```

Q9: Conflicting Signals

Trigger: Postdata vs Topspeed disagree + no consensus

Reason:

- Expert disagreement indicates uncertainty
- No clear pattern

Example:

```
race = {  
    'postdata_pick': 'Horse A',  
    'topspeed_pick': 'Horse B',  
    'consensus': False,  
}  
# Q9 TRIGGERED → QUARANTINE
```

Quarantine Decision Logic

```
def check_quarantine(race_data):  
    # Q5: Chaos Mode  
    if race_data['going'] in ['HEAVY', 'SOFT'] and race_data['runners'] >=  
12:  
        return {  
            'quarantine': True,  
            'gate': 'Q5',  
            'reason': 'Chaos Mode: Heavy/soft going + large field'  
        }  
  
    # Q6: Small Field  
    if race_data['runners'] < 5:  
        return {  
            'quarantine': True,  
            'gate': 'Q6',  
            'reason': 'Small field: Form unreliable'  
        }  
  
    # Q7: No Form Data  
    if race_data['race_type'] == 'MAIDEN' and race_data['runners_with_form']  
== 0:  
        return {  
            'quarantine': True,  
            'gate': 'Q7',  
            'reason': 'No form data: Maiden race'  
        }  
  
    # Q8: Market Chaos  
    if race_data['favourite_odds'] > 4.0 or race_data['odds_volatility'] >  
20:  
        return {  
            'quarantine': True,  
            'gate': 'Q8',  
            'reason': 'Market chaos: No clear favourite or high volatility'  
        }  
  
    # Q9: Conflicting Signals  
    if not race_data['consensus'] and race_data['postdata_pick'] !=  
race_data['topspeed_pick']:  
        return {  
            'quarantine': True,  
            'gate': 'Q9',  
            'reason': 'Conflicting signals: Expert disagreement'
```

```
    }  
  
    return {  
        'quarantine': False,  
        'gate': None,  
        'reason': 'All quarantine gates passed'  
    }
```

Quarantine Statistics (18 Feb 2026)

Total races processed: 20

Quarantines: 9 (45%)

By meeting:

- **Punchestown:** $\frac{4}{7}$ (57%) — Heavy going
- **Ludlow:** $\frac{3}{7}$ (43%) — Soft going
- **Southwell:** $\frac{2}{6}$ (33%) — AW standard

Key insight: Going conditions are the primary quarantine driver.

LAYER 8: FINAL DECISION POLICY

How VÉLØ Makes the Final Call

```
FEATURE ENGINES → p(win), p(top4)
  ↓
RPD-C TAGS → T/H/P/S/E classification
  ↓
MARKET CONSTRAINT → Favourite dismissed or respected
  ↓
INTENT DETECTION → Intent score (0-100)
  ↓
SCENARIO EVIDENCE → Scenario validated or rejected
  ↓
QUARANTINE GATES → Pass or quarantine
  ↓
FINAL VERDICT → Top Strike / Value / Danger / Suppress
```

Decision Logic (Complete)

```
def generate_verdict(race_data):
    runners = race_data['runners']

    # Step 1: Check quarantine gates
    quarantine_check = check_quarantine(race_data)
    if quarantine_check['quarantine']:
        return {
            'verdict': 'QUARANTINE',
            'gate': quarantine_check['gate'],
            'reason': quarantine_check['reason']
        }

    # Step 2: Filter by RPD-C tags
    targets = [r for r in runners if r['rpd_tag'] == 'T']
    holds = [r for r in runners if r['rpd_tag'] == 'H']
    candidates = targets + holds

    if len(candidates) == 0:
        return {
            'verdict': 'SUPPRESS',
            'reason': 'No valid targets or holds'
        }

    # Step 3: Market constraint check
    favourite = get_favourite(runners)
    if favourite and should_dismiss_favourite(favourite):
        candidates = [r for r in candidates if r != favourite]
        if len(candidates) == 0:
            return {
                'verdict': 'SUPPRESS',
                'reason': 'Favourite dismissed, no other candidates'
            }

    # Step 4: Calculate combined score for each candidate
    for runner in candidates:
        # Weighted combination of signals
        runner['combined_score'] = (
            runner['p_win'] * 0.35 + # Core predictor win probability
            (runner['intent_score'] / 100) * 0.25 + # Trainer intent
            runner['scenario_evidence'] * 0.20 + # Scenario validation
            (runner['bsp_advantage'] / 100) * 0.10 + # Market value
            (runner['form_consistency_score'] / 100) * 0.10 # Form
            reliability
```



```

    )

# Step 5: Rank by combined score
candidates.sort(key=lambda r: r['combined_score'], reverse=True)

# Step 6: Generate verdict
top_strike = candidates[0]
danger = candidates[1] if len(candidates) > 1 else None
value = candidates[2] if len(candidates) > 2 else None

# Step 7: Calculate confidence
confidence = calculate_confidence(top_strike)

return {
    'verdict': 'STRIKE',
    'top_strike': {
        'name': top_strike['name'],
        'odds': top_strike['odds_decimal'],
        'combined_score': top_strike['combined_score'],
        'p_win': top_strike['p_win'],
        'intent_score': top_strike['intent_score'],
        'scenario': top_strike['scenario'],
        'rpd_tag': top_strike['rpd_tag'],
    },
    'danger': {
        'name': danger['name'],
        'odds': danger['odds_decimal'],
    } if danger else None,
    'value': {
        'name': value['name'],
        'odds': value['odds_decimal'],
    } if value else None,
    'confidence': confidence,
    'rationale': generate_rationale(top_strike)
}

```

Confidence Calculation

```
def calculate_confidence(runner):
    confidence = 50 # Baseline

    # Factor 1: Win probability
    if runner['p_win'] > 0.35:
        confidence += 20
    elif runner['p_win'] > 0.25:
        confidence += 10

    # Factor 2: Intent score
    if runner['intent_score'] > 65:
        confidence += 15
    elif runner['intent_score'] > 50:
        confidence += 10

    # Factor 3: Scenario validation
    if runner['scenario_evidence'] > 0.8:
        confidence += 15
    elif runner['scenario_evidence'] > 0.6:
        confidence += 10

    # Factor 4: Form consistency
    if runner['form_consistency_score'] < 5:
        confidence += 10

    # Factor 5: Market validation
    if runner['bsp_advantage'] > 15:
        confidence += 10

    # Cap at 100
    return min(confidence, 100)
```

Confidence Interpretation

Confidence	Interpretation	Action
80-100	HIGH	Strong strike recommendation
60-79	MEDIUM	Conditional strike
40-59	LOW	Weak strike, monitor only
<40	VERY LOW	Suppress or quarantine



THE DOCTRINE: WHY VÉLØ WORKS

The Three Pillars

1. Truth Before Optimization

Most systems optimize for:

- Strike rate (% of winners picked)
- ROI (return on investment)
- User engagement (picks that sound good)

VÉLØ optimizes for:

- **Truth** (is this actually the best horse?)
- **Evidence** (can I prove it?)
- **Confidence** (am I sure?)

Result: VÉLØ quarantines 45% of races because **it's honest about uncertainty.**

Example:

```
# Traditional system
if p_win > 0.20:
    return "STRIKE" # 20% threshold, high strike rate

# VÉLØ
if p_win > 0.25 and intent_score > 50 and scenario_valid and confidence > 60:
    return "STRIKE" # Multiple gates, lower strike rate but higher accuracy
else:
    return "QUARANTINE" # Honest about uncertainty
```

2. Memory Before Learning

Most systems:

- Train on historical data
- Deploy model
- Never look back

VÉLØ:

- Logs every prediction
- Runs post-race SIGMA evaluation
- Identifies systematic errors (like American State)
- Updates tagging logic
- **Learns from mistakes**

Example: American State Learning Loop

```
PREDICTION: Mr Nugget (Top Strike)
ACTUAL: American State won @ 17.0
↓
SIGMA EVALUATION
↓
IDENTIFIED: P-tag misclassification
↓
PROPOSED: R tag (Reactivation) for wind surgery + new yard + BSP advantage
↓
UPDATED: RPD-C v2 logic
↓
NEXT RACE: American State pattern recognized correctly
```

3. Doctrine Before Power

Most systems:

- Add more features
- Train bigger models
- Chase marginal gains

VÉLØ:

- Follows strict doctrine (5 filters, quarantine gates)
- Refuses to compromise on evidence
- **Discipline over complexity**

The 5 Filters:

1. **No noise** — Only signal (RPD-C tags filter noise)
 2. **No emotion** — Only logic (algorithmic decision-making)
 3. **No assumptions** — Only evidence (scenario validation)
 4. **No drifting** — Only focus (quarantine gates maintain discipline)
 5. **No illusions** — Only truth (honest confidence scores)
-



COMPLETE EXAMPLE: RACE WALKTHROUGH

Race: Wolverhampton 19:30, 1m1f Handicap, 10 runners, Standard (AW)

Step 1: Feature Engines Run

61 features calculated for each of 10 runners.

Example: Mr Nugget

```
features = {  
    # Form  
    'rpr_last_3_avg': 80,  
    'ts_last_3_avg': 75,  
    'or_current': 78,  
    'form_consistency_score': 3.2,  
    'peak_form_recency': 14,  
    'career_win_rate': 0.22,  
    'c_d_win_rate': 0.40,  
  
    # Pace  
    'early_pace_score': 68,  
    'late_pace_score': 72,  
    'pace_geometry': 0.55,  
  
    # Draw  
    'draw_bias_score': 15,  
    'draw_advantage_index': 0.90,  
    'rail_position_flag': 1,  
  
    # Trainer/Jockey  
    'trainer_strike_rate_14d': 0.20,  
    'jockey_strike_rate_14d': 0.23,  
    'first_choice_jockey_flag': 1,  
    'trainer_intent_score': 72,  
    'stable_form_cycle': 68,  
  
    # Course/Going/Distance  
    'course_form_score': 85,  
    'going_suitability_score': 90,  
    'distance_win_rate': 0.35,  
    'trip_suitability_index': 88,  
  
    # Class  
    'class_rating': 4,  
    'class_movement': 0,  
    'or_vs_class_gap': +5,  
  
    # Recency  
    'days_since_last_run': 21,  
    'peak_fitness_window': 1,  
  
    # Market  
    'odds_decimal': 2.5,  
    'odds_drift_30m': -5.2,
```

```
'bsp_advantage': -5,  
}
```

Step 2: Core Predictor Outputs

LightGBM model processes 61 features:

```
# Model prediction  
p_win = 0.35 # 35% win probability  
p_top4 = 0.72 # 72% top-4 probability
```

All 10 runners:

Horse	p(win)	p(top4)
Mr Nugget	0.35	0.72
American State	0.12	0.45
Corundum	0.18	0.58
How's The Guvnor	0.15	0.52
Stable Genius	0.08	0.35
Fast Eddie	0.06	0.28
Lucky Strike	0.04	0.22
Midnight Runner	0.03	0.18
Golden Boy	0.02	0.15
Last Chance	0.01	0.10

Step 3: RPD-C Tags

Mr Nugget:


```

positive_signals = [
    'rpr_peak_form', # RPR 80 near best
    'first_choice_jockey', # ✓
    'c_d_winner', # ✓ 40% C&D win rate
    'good_draw', # ✓ Stall 1
    'stable_in_form', # ✓ 68% stable form
]
# 5 positive signals → Tag = T (Target)

```

American State:

```

prep_indicators = [
    'long_layoff', # ✓ 70 days
    'wind_surgery', # ✓
    'new_yard', # ✓
]
# 3 prep indicators → Tag = P (Prep)
# SHOULD BE: P2 (Reactivation) due to BSP advantage +19%

```

All 10 runners:

Horse	Tag	Reason
Mr Nugget	T	5 positive signals
American State	P	3 prep indicators (MISSED: should be P2)
Corundum	T	4 positive signals
How's The Guvnor	T	4 positive signals
Stable Genius	H	3 positive signals
Fast Eddie	S	2 negative signals
Lucky Strike	S	2 negative signals
Midnight Runner	E	2 hard elimination signals
Golden Boy	E	2 hard elimination signals
Last Chance	E	2 hard elimination signals

Candidates: Mr Nugget, Corundum, How's The Guvnor, Stable Genius

Step 4: Market Constraint

Favourite: Mr Nugget (2.5)

```
bsp_drift = ((2.5 - 2.6) / 2.6) * 100 = -3.8%
# Drift < 15% → PASS

counter_signals = [
    'hat_trick_bid_pressure', # Slight pressure
]
# 1 counter-signal (need 3 for dismissal) → PASS

verdict = "RESPECT_FAVOURITE"
```

Step 5: Intent Detection

Mr Nugget:

```
intent_score = (
    20 + # First-choice jockey
    15 + # Optimal trip
    10 + # C&D winner
    10   # Stable in form
) = 55 # MODERATE INTENT
```

All candidates:

Horse	Intent Score
Mr Nugget	55
Corundum	50
How's The Guvnor	48
Stable Genius	40

Step 6: Scenario Evidence

Mr Nugget:

```
scenario = 'S1' # Hat-trick bid
required_evidence = [
    'won_last_2', # ✓
    'same_class', # ✓
    'same_trip', # ✓
    'within_30_days', # ✓
]
evidence_ratio = 4/4 = 1.0 # 100% → VALID
```

All candidates:

Horse	Scenario	Evidence Ratio
Mr Nugget	S1 (Hat-trick)	1.0 (VALID)
Corundum	S3 (Class drop)	0.75 (VALID)
How's The Guvnor	S2 (Course specialist)	0.75 (VALID)
Stable Genius	None	0.0

Step 7: Quarantine Check

```
race_data = {  
    'going': 'STANDARD', # AW  
    'runners': 10,  
    'race_type': 'HANDICAP',  
    'favourite_odds': 2.5,  
    'consensus': True, # Postdata + Topspeed agree  
}  
  
# Q5: Chaos Mode → NO (standard going)  
# Q6: Small Field → NO (10 runners)  
# Q7: No Form Data → NO (handicap with form)  
# Q8: Market Chaos → NO (clear favourite)  
# Q9: Conflicting Signals → NO (consensus)  
  
verdict = "PASS" # No quarantine
```

Step 8: Combined Score Calculation

Mr Nugget:

```
combined_score = (  
    0.35 * 0.35 + # p_win  
    (55 / 100) * 0.25 + # intent_score  
    1.0 * 0.20 + # scenario_evidence  
    (5 / 100) * 0.10 + # bsp_advantage (negative, so low)  
    (3.2 / 100) * 0.10 # form_consistency  
) = 0.123 + 0.138 + 0.200 + 0.005 + 0.003 = 0.469
```

All candidates:

Horse	Combined Score
Mr Nugget	0.469
Corundum	0.385
How's The Guvnor	0.368
Stable Genius	0.285

Step 9: Final Verdict

Top Strike: Mr Nugget (0.469)

Danger: Corundum (0.385)

Value: How's The Guvnor (0.368)

Confidence:

```
confidence = (  
    50 + # Baseline  
    20 + # p_win > 0.35  
    10 + # intent_score > 50  
    15 + # scenario_evidence > 0.8  
    10  # form_consistency < 5  
) = 105 → capped at 100
```

Final Output:

```
{
  "verdict": "STRIKE",
  "top_strike": {
    "name": "Mr Nugget",
    "odds": 2.5,
    "combined_score": 0.469,
    "p_win": 0.35,
    "intent_score": 55,
    "scenario": "S1 (Hat-trick bid)",
    "rpd_tag": "T"
  },
  "danger": {
    "name": "Corundum",
    "odds": 5.5
  },
  "value": {
    "name": "How's The Guvnor",
    "odds": 7.0
  },
  "confidence": 100,
  "rationale": "Mr Nugget attempting hat-trick with strong C&D record, first-choice jockey, optimal draw (stall 1), and stable in form. Scenario S1 (Hat-trick bid) fully validated. High confidence strike."
}
```

Step 10: Actual Result

Winner: American State @ 17.0

Post-race SIGMA:

- Mr Nugget finished 2nd  (top-4 containment)
- Corundum finished 3rd  (top-4 containment)
- How's The Guvnor finished 4th  (top-4 containment)
- American State finished 1st  (missed due to P-tag misclassification)

Learning:

- P tag should be split into P1 (Prep) and P2 (Reactivation)

- **BSP advantage >15% for P-tagged horses = informed money recognizing reactivation**
- **Wind surgery + new yard + break <120 days = reactivation pattern, not prep**

Updated logic:

```
if runner['rpd_tag'] == 'P' and runner['bsp_advantage'] > 15:  
    runner['rpd_tag'] = 'P2' # Reactivation  
    runner['consider_for_strike'] = True
```

SUMMARY

How VÉLØ Actually Works

1. **61 feature engines** extract signals from raw data
2. **Core predictor (LightGBM)** outputs win/top-4 probabilities using gradient boosting
3. **RPD-C tags** classify runners (T/H/P/S/E) based on evidence patterns
4. **Market Constraint Engine** detects fake favourites via BSP drift analysis
5. **Intent Detection** identifies trainer “go days” using 13 signals
6. **Scenario Evidence Gate** validates race-winning patterns (8 scenarios)
7. **Quarantine Gates** refuse to bet when uncertainty is high (5 gates)
8. **Final Decision Policy** combines all layers into strike recommendation
9. **Post-race SIGMA** learns from mistakes and updates logic

Why VÉLØ Works

Truth before optimization — Honest about uncertainty (45% quarantine rate)

Memory before learning — Learns from mistakes (American State → R tag)

Doctrine before power — Discipline over complexity (5 filters, strict gates)

The Edge

VÉLØ doesn't predict horses. It predicts **people pretending to be unpredictable**.

- **Trainers** have intent patterns
- **Markets** have manipulation patterns
- **Races** have scenario patterns
- **Form** has context patterns

VÉLØ codifies the intangible. This is what makes it feel alive.

Truth before optimization. Memory before learning. Doctrine before power.

VÉLØ Oracle Prime — Phase 1 Build

Date: 18 February 2026